

Jonathan Brugh

Enhancement One Narrative

08/08/2026

Overview

The original artifact is a console program that reads a CSV file of course data into a binary search tree (BST) and lets a user search for and display course information. It works, but the code review makes it clear that it was a first pass. The code review added placeholder notes which effectively documents the gap between what the program does and what a production-quality version of it should do. This enhancement is the direct response to that self-assessment. It keeps the same core data structure and algorithmic approach, which is a BST keyed on courseId, but closes nearly every gap the code review of the original artifact identified, and in doing so demonstrates the software design and engineering competencies the capstone requires to show: defensive programming, separation of concerns, meaningful documentation, and design that anticipates how real users misuse a program.

From a Fragile Program to a Defensive One

The clearest engineering improvement is how this enhancement treats user input as untrustworthy by default. In the original artifact, loadCSV() simply calls file.open(csvPath) and starts reading. If the path is wrong, the ifstream fails quietly and the program appears to do nothing, offering the user no indication of what went wrong or how to recover. This is a common symptom of code written to satisfy the happy path rather than the real one.

Enhancement One replaces that assumption with an explicit check. Before any read is attempted, `std::filesystem::exists(csvPath)` confirms the file is actually there. If it is not, the program does not fail silently, but rather tells the user, gives them up to three chances to enter a corrected file name, and re-validates each attempt against both existence and the .csv extension using a new `checkCSV()` helper. Only after those checks pass does the program open the stream. This is a textbook illustration of defensive programming. The code assumes external input will be wrong at some point and builds a recovery path for that case instead of letting the failure propagate silently to the user.

The same philosophy appears in `main()`. The original artifact clears `cin`'s fail state after bad input but takes no further action, meaning an out-of-range integer simply falls through the switch statement with no feedback. This enhancement adds a default case that catches any unrecognized selection and prompts the user again, closing a gap between what the interface implies is possible and what the code actually guards against.

Correcting a Data-Integrity Bug

A subtler but important fix involves how prerequisites are parsed from the CSV. In the original artifact's `loadCSV()`, the loop that reads remaining comma-separated fields pushes every token onto the `preReqs` vector, including empty strings produced by trailing commas or blank fields. There is no branch to handle the case where a field is empty, and the code review comment flags this as a known, unresolved issue. The practical effect is that a course with no prerequisites can end up with an inconsistent or misleading internal representation depending on exactly how its CSV row is formatted.

Enhancement One resolves this directly: the parsing loop now checks each token, and if it is empty, pushes the literal string "None" instead of an empty string. This is a small change with an outsized design implication. It means the Course object's state is always well-defined and self-describing, rather than depending on the display logic downstream to paper over gaps in the data. That is the difference between patching a symptom at the point where it becomes visible, as the original's display functions had to do with conditional checks for empty vectors, and fixing the root cause at the point where the data enters the system.

Cleaner Interfaces and Reduced Surface Area

The original BinarySearchTree class declares PostOrder(), PreOrder(), postOrder(), and preOrder(), and the code review comments note they are not called and should be removed.

Carrying unused public methods is a maintenance liability, as every method in a public interface is something future developers have to read, reason about, and assume is meaningful.

Enhancement One removes all four, leaving a class whose public surface, InOrder(), Insert(), and Search(), maps exactly to the operations the program uses. This reflects a core software design principle that an interface should expose what a caller needs and nothing more, both to reduce cognitive load and to shrink the amount of code that has to be tested and maintained.

The console interface received a parallel cleanup. In the original, the menu is printed as a handful of inline cout statements buried in main(), making it difficult to reuse or modify independently of the program's control flow. This enhancement extracts this into two dedicated functions, userMenu() and topBottomMenuBorder(), so the menu's presentation is decoupled from the logic that drives it. This is a small-scale example of separation of concerns: main() is now responsible

for orchestrating program flow, while presentation details live in functions built specifically for that purpose.

Documentation as a Design Artifact, Not an Afterthought

The code review of the original artifact identified areas where descriptive commenting was lacking. In Enhancement One, those same locations now contain comments that describe actual purpose and behavior, such as the `Course` struct's comment explaining that its fields are used by the `BST` class and specifically calls out that `courseId` drives placement; the `BinarySearchTree` class comment states plainly that it "holds the CRUD operations and builds the BST in memory"; and the file itself opens with a block comment explaining what the whole program does, something the original never states.

This shift matters for more than tidiness. Comments that describe TODOs and comments that describe design decisions serve fundamentally different audiences. The former talks to the original author's future self, the latter talks to any engineer inheriting the code. Enhancement One is written for the second audience, which is a hallmark of engineering maturity, where code is treated as a durable artifact meant to be read and maintained by others and not just a working script to be revisited by the person who wrote it.

Summary Comparison

Aspect	Original Artifact	Enhancement One
Namespace usage	using namespace std; applied globally, risking symbol collisions.	using namespace std; removed; every standard-library type is explicitly scoped (std::string,

Aspect	Original Artifact	Enhancement One
		std::cout, etc.).
File loading	Assumes the path is valid; a bad path silently fails with no feedback.	Verifies the file exists with std::filesystem::exists, then gives the user up to three attempts to supply a valid .csv path before returning to the menu.
Input validation	Menu accepts any file path without checking the extension.	checkCSV() confirms a .csv extension before attempting to load, and re-validates during the retry loop.
Missing prerequisites	Loop silently skips empty prerequisite fields, so a course can print fewer entries than the header intends and no 'no prerequisite' state is recorded.	Empty fields are explicitly captured and stored as "None" in the vector, so the data model always reflects the true state of a course.
Documentation	Comments are inconsistent: some are TODO-style reminders, several methods have no header comments at all.	Every class, struct, and non-trivial method carries a purpose-driven comment or Doxygen-style header; TODO markers are resolved into

Aspect	Original Artifact	Enhancement One
		working code or intentionally deferred with a clear rationale.
Dead code	PostOrder/PreOrder traversals and their declarations are implemented but never invoked.	Unused traversal methods are removed entirely, shrinking the public interface to what the program actually uses.
User interface	Menu is printed inline in main() with a hard-coded switch and no visual structure.	Menu is extracted into userMenu() and topBottomMenuBar(), producing a bordered, reusable, testable UI component; ANSI color improves console readability.
Error handling	A single cin.fail() check clears bad input but does not confirm the chosen case exists in the switch statement.	Adds a default case to the switch so any out-of-range integer is handled gracefully instead of silently doing nothing.
Code organization	Struct and class comments are missing. An overall description of the program is missing.	Struct and class comments and an overall program description describe actual purpose and

Aspect	Original Artifact	Enhancement One
		usage, reflecting a completed design rather than a to-do list.

What This Enhancement Demonstrates

Taken together, these changes show more than incremental bug fixing. They show an engineering process by identifying weaknesses through structured self-review, which is visible in the original artifact's code review embedded comments, prioritize the ones that affect correctness and user trust, and address them with techniques appropriate to each problem, including input validation and existence checks for I/O robustness, root-cause data correction for integrity, interface reduction for maintainability, and rewritten documentation for long-term readability. None of the underlying algorithm changed. The BST insertion, search, and in-order traversal logic are functionally the same tree operations as before. What changed is the engineering discipline wrapped around that algorithm, which is precisely the distinction between a script that runs once for a demo and software that is ready to be handed to another developer or deployed for real users.

The class's own comment in Enhancement One still notes that a remove/update capability remains future work. Leaving that gap explicitly documented, rather than silently dropped, is itself consistent with the enhancement's overall approach, which is to be honest in the code about what is done, what is deferred, and why.